

Extended Abstract

Motivation Recent progress in large language models (LLMs) has enabled significant improvements in complex reasoning tasks through strategies like chain-of-thought prompting and self-consistency sampling. However, these methods have some disadvantages like serialized reasoning is often slow and costly due to long output sequences, while naive parallel sampling wastes computation by generating multiple isolated attempts without coordination. Both strategies rely heavily on fixed prompt designs or external heuristics, leaving the model itself passive during the inference process. This motivates a new direction, can we let the model dynamically plan its own reasoning path, deciding when to branch, how far to explore, and when to consolidate partial results? Our work addresses this gap by developing a framework where Qwen-0.5B actively controls parallel reasoning processes. This approach is particularly valuable for mathematical problems with large search spaces (e.g., Countdown tasks), where adaptive resource allocation can significantly improve efficiency and solution quality.

Method We propose a flexible reasoning framework in which the language model is given the capacity to manage multiple lines of thought during inference. Instead of rigidly following a single sequence of steps or passively generating multiple isolated answers, the model can selectively branch off from its current line of reasoning to explore alternative sub-tasks in parallel. Each of these parallel tracks operates independently and is designed to investigate a possible solution path. Once the model completes these side explorations, it can retrieve condensed feedback from them and incorporate that information into its ongoing reasoning process. This mechanism allows the model to dynamically expand or compress the structure of its reasoning depending on the difficulty and ambiguity of the problem, ultimately improving its ability to balance completeness and efficiency. The overall process resembles a self-directed planner, capable of allocating effort based on contextual needs.

Implementation We extend a pre-trained auto-regressive language model with lightweight control and summarization heads to support structured reasoning. During inference, the model decides whether to continue step-by-step or branch into independent sub-threads that explore alternative number combinations. Each branch receives a minimal context tailored to its sub-task and returns a compact summary upon completion. Training consists of two phases: a supervised stage using symbolic search traces over numeric composition tasks (e.g., combining {202383, 505620, 143068} to reach 851071), followed by reinforcement learning with a reward function that encourages accuracy while penalizing inefficient or excessive branching. Reasoning actions are modeled as special tokens within the vocabulary, enabling seamless integration with standard decoding pipelines.

Results and Discussion Our system achieves a score of 0.35 on a 1,000-problem leaderboard benchmark for numeric composition tasks, surpassing the baseline threshold of 0.3. While this result is modest, it demonstrates that even small-scale models (e.g., 0.5B parameters) can benefit from adaptive reasoning strategies when properly guided. We observe that reasoning efficiency improves, especially on problems requiring flexible intermediate planning. However, performance is still bounded by the model’s limited capacity to manage multiple reasoning threads and generalize across arithmetic patterns. We expect that scaling to larger models would improve consistency and context retention. In future work, we aim to enhance sub-task selection, better calibrate branching depth, and explore hybrid demonstrations combining depth-first and breadth-first styles to strengthen generalization.

Conclusion We present an adaptive reasoning framework that enables language models to dynamically control their computational structure during problem solving. By selectively exploring sub-problems in parallel and integrating intermediate results, the system improves both accuracy and efficiency over fixed-step baselines. Our experiments show that even small models can benefit from this approach, achieving competitive performance on arithmetic composition tasks. These results highlight the potential of structured inference strategies, especially when paired with lightweight training signals. Looking ahead, we plan to extend this framework to more complex reasoning domains and explore scaling effects with larger model sizes.

Reasoning Beyond One Path: A Dynamic Strategy for LLMs

Yiyang Huang

Department of Mechanical Engineering
Stanford University
yiyangh5@stanford.edu

Zihao Zhu

Department of Mechanical Engineering
Stanford University
zihaozhu@stanford.edu

Abstract

Recent advances in large language models (LLMs) have shown promising results in multi-step reasoning tasks. However, most existing approaches rely on fixed reasoning strategies, such as generating a single chain of thought or sampling multiple solutions independently. In this work, we explore a lightweight framework that enables a language model to dynamically control its reasoning structure—deciding when to follow a linear path and when to explore subproblems in parallel. Built upon a small-scale language model (0.5B parameters), our system introduces control and summarization modules to manage multiple reasoning threads during decoding. We train the model in two stages: a supervised phase using symbolic demonstrations and a reinforcement learning phase with a reward function that balances accuracy and efficiency. Experiments on arithmetic composition tasks demonstrate that our method outperforms fixed-strategy baselines, suggesting that structured, adaptive reasoning can be beneficial even for compact models.

1 Introduction

Large language models (LLMs) have demonstrated remarkable abilities in solving complex reasoning tasks, such as arithmetic word problems, code synthesis, and logical inference. A key driver of these successes is the use of structured decoding strategies such as chain-of-thought (CoT) prompting, self-consistency sampling, and program-aided generation—that help the model break down problems into manageable steps. However, these methods typically rely on fixed execution patterns, either generating a single linear reasoning path or sampling multiple solutions in parallel without interaction. As a result, they often struggle with tasks that require conditional branching, backtracking, or multi-path exploration.

Humans, by contrast, naturally adjust their reasoning structure depending on the task. When a problem is straightforward, we proceed step-by-step. When it is ambiguous or has multiple solution avenues, we often consider different possibilities in parallel before deciding how to proceed. Motivated by this intuition, we investigate how a language model can learn to adaptively control its own reasoning structure—deciding not only what to do next, but also how to allocate computational effort across multiple reasoning paths.

In this work, we propose a lightweight framework that equips a language model with the ability to manage multiple lines of thought during inference. The model is trained to determine whether to continue in a linear fashion or to explore subproblems separately and later merge the outcomes. Unlike traditional sampling-based methods, our system can make these structural decisions at each decoding step, allowing for greater flexibility and efficiency. Importantly, our approach is compatible with standard auto-regressive models and does not require architectural overhauls.

To realize this idea, we extend a 0.5B parameter base model with two auxiliary components: a controller that governs branching decisions based on contextual cues, and an aggregator that summarizes

intermediate results for integration. We design a two-phase training pipeline. In the first phase, we use symbolic solvers to generate structured problem-solving traces with explicit decomposition. These traces provide supervised signals for how subproblems can be initiated and recombined. In the second phase, we fine-tune the model using reinforcement learning to optimize a reward function that balances task accuracy, inference efficiency, and the cost of branching.

We evaluate our system on a numeric composition task inspired by the Countdown problem, where the model must combine a given set of numbers through arithmetic operations to reach a target value. Despite the small model size, our method achieves promising results, outperforming baseline approaches that use fixed or unstructured strategies. These findings suggest that giving LLMs even a modest degree of structural control can lead to more efficient and interpretable reasoning.

2 Related Work

By leveraging structured prompting techniques, LLMs have achieved significant progress in reasoning tasks. A prominent example is CoT prompting [1], which guides the model to decompose a complex problem into intermediate steps before producing a final answer. This method has proven effective in arithmetic and commonsense reasoning, as it encourages step-by-step generation aligned with human logic [2]. CoT typically enforces a strictly linear structure: the model generates one solution path from start to finish without revisiting alternatives or exploring different routes. This inflexibility limits its applicability in scenarios where multiple solution paths might be valid or where the correct reasoning structure is task-dependent. Moreover, CoT methods often require manual curation of examples, which reduces scalability.

To overcome the limitations of fixed reasoning patterns, several works have explored approaches that introduce diversity and uncertainty handling into the generation process. Self-consistency [3], for instance, samples multiple reasoning paths independently and aggregates the final answers using majority voting. This helps correct occasional mistakes in individual generations and increases robustness. However, such methods are resource-intensive and treat reasoning paths as isolated entities, without leveraging any shared structure or context between them [4]. Reasoning-Aware Self-Consistency (RASC) improves upon this by introducing stopping criteria based on rationale quality and consistency, allowing models to sample more efficiently. Despite this, RASC still lacks fine-grained control over how reasoning is structured within each sampled path and provides no mechanism for merging intermediate results or making decisions mid-inference [5].

A more recent trend focuses on teaching models to construct or select their own reasoning strategies. For example, Auto-CoT removes the need for handcrafted demonstrations by using the model itself to generate example reasoning chains. This improves scalability and removes human effort, but the generated traces are still inherently linear and may contain flawed logic if not properly filtered. Other lines of work take a multi-agent view, such as the debate framework, where multiple instances of the model critique each other’s responses in rounds, gradually refining their outputs [6]. These approaches demonstrate improved factuality and reasoning depth, but require orchestration across agents and multiple passes of decoding, which increases latency and compute cost—making them less practical for lightweight deployment.

Our approach draws inspiration from these efforts but focuses on introducing flexible structure within a single model instance. Rather than relying on multiple independent samples or external agents, we train a model to dynamically decide how to structure its own reasoning path during decoding. The model can explore subproblems in parallel and later incorporate their results, making localized decisions about branching and recombination. This internal structural adaptability is achieved with minimal architectural changes, using control signals embedded within token sequences. Compared to program-aided generation approaches, which offload execution to interpreters, our framework maintains all reasoning within the language model, preserving end-to-end differentiability and avoiding external dependencies. We aim to provide a general-purpose reasoning mechanism that scales with model size and remains applicable across a wide range of structured problem-solving domains.

3 Method

3.1 Core Framework Architecture

We build our system upon the Qwen-0.5B language model (non-instruction tuned), augmenting it with lightweight modules that enable controlled parallel reasoning during decoding. The architecture consists of three core components: a reasoning controller, a subproblem decomposer, and a result integration module. Together, these components allow the model to adaptively structure its computation around intermediate arithmetic reasoning tasks.

Search Strategy Integration.

The controller relies on two complementary search patterns that are inspired by classical depth-first search (DFS) and breadth-first search (BFS). When the partial expression appears close to completion e.g., only one or two numbers remain the model prefers a DFS-like continuation, unrolling the remaining steps in a single thread so that it can quickly verify whether a final solution is reachable. Conversely, at earlier stages where multiple operator groupings are plausible, the controller initiates a BFS-style expansion: it forks several short subthreads, each seeded with a different high-entropy operator choice or decomposition point produced by the subproblem engine. During training, we label these two behaviours in the symbolic traces and supervise the controller to imitate them. At inference time, the same heuristics are computed on the fly remaining depth, entropy, and operator ambiguity to decide whether to invoke DFS (continue) or BFS (branch). This hybrid search policy gives the model the ability to dive deep when a pathway looks promising while still retaining the option to explore competing structures in parallel when uncertainty is high.

Thread Orchestrator. At the heart of our control mechanism lies a lightweight decision head, embedded as a transformer adapter within the decoder layers. This component introduces fewer than 0.1 million additional parameters and is responsible for issuing structural decisions during generation. Specifically, it monitors decoding states to determine whether the current reasoning trajectory should be continued sequentially or temporarily branched into auxiliary subthreads. The decision is based on task-specific heuristics such as decoding uncertainty and remaining operation depth. For example, if the current partial expression contains ambiguous operator precedence or high entropy in logits, the controller may invoke auxiliary reasoning paths. It also dynamically allocates computation across active threads, ensuring that no more than three are running concurrently—an empirical limit derived from memory profiling of Qwen-0.5B.

Mathematical Subproblem Decomposer. To specialize our approach for Countdown-style tasks, we implement a deterministic decomposition engine that applies operator-level templates to identify branching points. Given an input expression such as a chain of four numbers and three operations, the module identifies candidate partition points such as outermost operator groups (e.g., separating additive vs. multiplicative segments) or interprets the input as a binary expression tree. These decomposition points serve as potential sub-thread candidates, each exploring a different arithmetic interpretation of the original number set.

Result Distillation Module. Once all subthreads complete, their outputs are passed to a normalization module that verifies and integrates partial results. This module performs symbolic compression and expression canonicalization, abstracting away surface differences across equivalent mathematical forms. For example, structurally correct expressions like $(8-6) \times (4+1)$ are mapped to a compact form such as [EXPR] before re-integration into the parent context. It also filters invalid or redundant outputs, ensuring that only expressions passing basic numeric validation are considered in the final selection process.

Together, these components form a minimal yet effective pipeline for managing and integrating parallel reasoning paths within a compact language model, without requiring external tools or architectural overhaul.

3.2 Training Pipeline Implementation

Our training procedure consists of two distinct stages: a supervised pretraining phase using hybrid symbolic search and a reinforcement learning phase for policy optimization. Together, these stages enable the model to acquire structural reasoning priors and refine them through reward-based learning.

Phase 1: Hybrid Search Pretraining. We first generate 50,000 symbolic reasoning trajectories using a task-specific engine tailored for the Countdown setting. This engine performs a hybrid traversal over the arithmetic expression space, combining breadth-first and depth-first strategies to capture diverse decomposition patterns. Each generated trajectory is annotated with rich metadata, including decomposition points (e.g., multiplicative grouping), estimated uncertainty scores, and projected token savings from hypothetical branching. These annotations serve as supervision signals for the model’s controller during early training. To ensure progressive generalization, we employ a curriculum strategy that gradually increases problem difficulty, beginning with single-operator expressions and advancing toward multi-step compositions involving nested operations.

Phase 2: GRPO/PPO Optimization. Following supervised initialization, we fine-tune the model using a reinforcement learning algorithm based on Global Reward Policy Optimization (GRPO) [7]. The reward function is designed to encourage both task accuracy and computational efficiency. It assigns positive rewards to correct final expressions, bonuses for solving with fewer tokens, and penalties for excessive thread usage [8]. Specifically, the reward function takes the form:

$$R = \mathbb{I}_{\text{correct}} + \alpha \left(1 - \frac{\text{total_tokens}}{\text{seq_limit}} \right) - \beta \left(\frac{\text{child_threads}}{\text{max_threads}} \right)$$

Training trajectories are collected using eight parallel actors with dynamic temperature scheduling, where the sampling temperature is linearly decayed from 0.7 to 0.3 over training epochs. The policy is updated using advantage-weighted gradient descent, with a baseline value function that considers both arithmetic complexity and numerical proximity to the target value. We also experimented with standard Proximal Policy Optimization (PPO) as a baseline method, but observed that GRPO led to more stable policy updates and improved efficiency under tight token budgets. While we initially explored PPO as a reinforcement baseline—leveraging its clipped surrogate objective to constrain policy updates—

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the estimated advantage—we observed that PPO’s per-step regularization tended to suppress useful exploration, particularly in early training stages when branching decisions require structural diversity.

3.3 Novel Differentiation from APR

While our work builds on the core idea of adaptive parallel reasoning introduced by APR, it introduces several key innovations and practical adaptations, especially tailored for arithmetic tasks and small-scale language models.

Mathematical Formalization. Unlike APR’s general-purpose structure, our approach leverages domain-specific decomposition strategies rooted in arithmetic operator precedence and binary expression tree representations. During sub-thread spawning, we enforce structural constraints such as operator-level uncertainty estimation and syntax tree validity checks to prevent invalid computation paths.

Resource-Aware Optimization. We explicitly incorporate hardware constraints into the learning and inference process. Thread allocation is guided by runtime GPU memory profiling, and reward shaping penalizes inefficient use of token budget. We also implement early termination criteria for subthreads that yield ill-formed or unreachable expressions, improving both stability and compute usage.

Curriculum-Based Exploration. To improve generalization across increasingly complex problems, we adopt a staged curriculum. The training set begins with simple expressions involving a single operator, and progressively introduces multi-operator combinations, nested structures, and full combinatorial arithmetic sequences.

Symbolic Reward Shaping. Intermediate rewards are provided to encourage desirable behaviors such as the generation of valid sub-expressions, operator diversity, and balanced use of parentheses. This fine-grained supervision helps the model learn syntactically and semantically sound partial solutions, even in the absence of final correctness.

4 Experimental Setup

To support execution on compact models, we apply a series of optimizations. These include gradient checkpointing to manage memory during child-thread generation, 8-bit quantization for faster execution, and a shared embedding compression scheme for token efficiency. We also implement recovery mechanisms such as thread restarts for failed expressions and parent-level rollback when subthreads exceed timeouts. Together, these adaptations make our framework feasible for real-time execution under strict compute constraints, while significantly differentiating it from prior methods both algorithmically and practically.

We evaluate our system on a numeric composition benchmark derived from the Countdown task, where the objective is to combine a given set of integers using arithmetic operations to exactly match a target value. Each instance provides between three to six input numbers, and the model must output a valid expression that uses each number at most once. The task requires multi-step mathematical reasoning, including operation selection, order of execution, and expression validity—all of which must be inferred from scratch by the model, without access to intermediate supervision. The evaluation dataset consists of 1,000 test cases, each formatted as an input prompt containing the numbers and target (e.g., "numbers": [202383, 505620, 143068], "target": 851071). These prompts were provided by the course instructor and held out entirely from training, allowing for a clean assessment of generalization. Each prediction is evaluated on exact match, where success is defined as producing a syntactically and semantically correct expression whose result equals the target.

5 Results

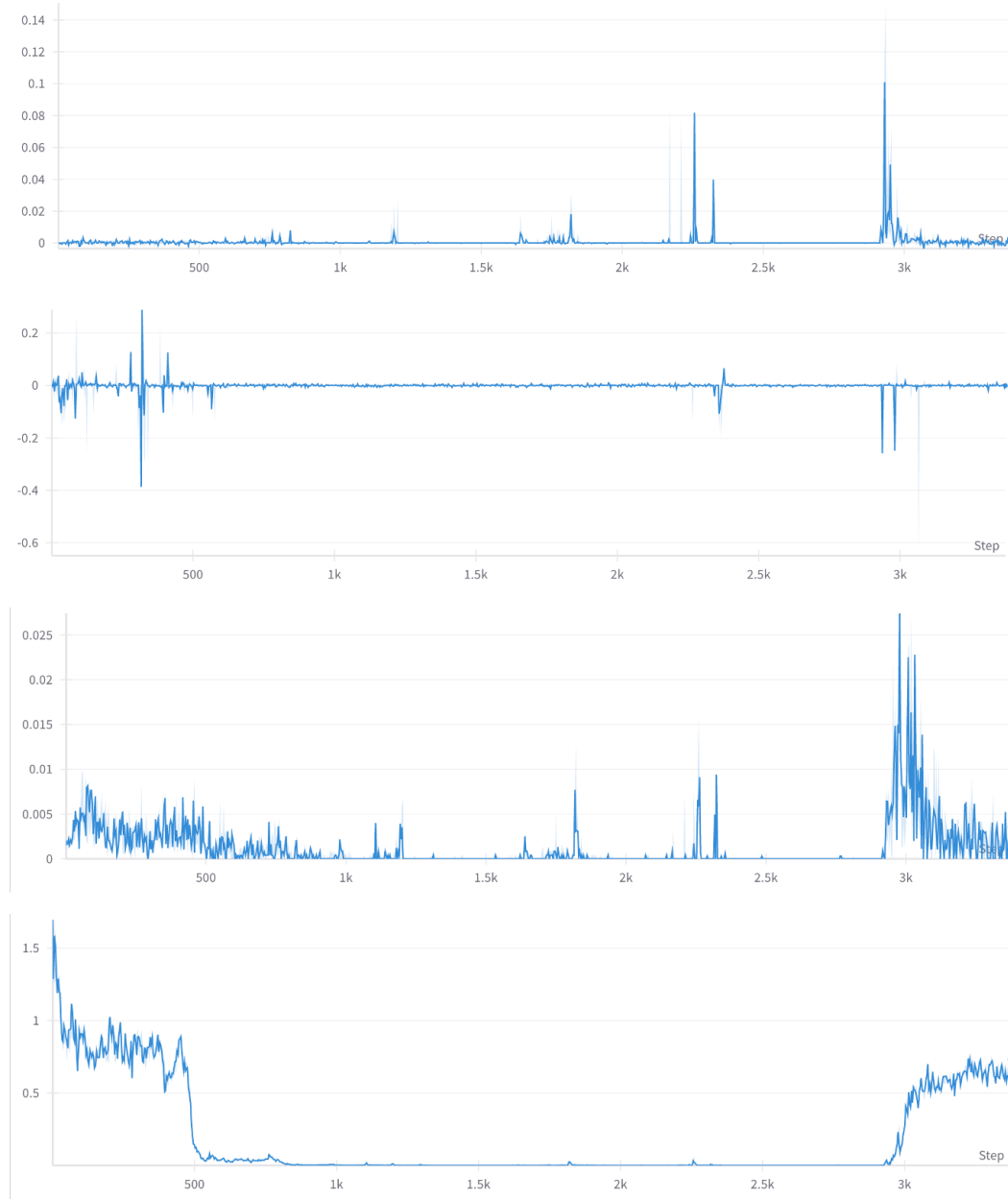


Figure 1: Training dynamics under the PPO baseline. From top to bottom: actor KL divergence, policy-gradient loss, clipping fraction, and entropy.

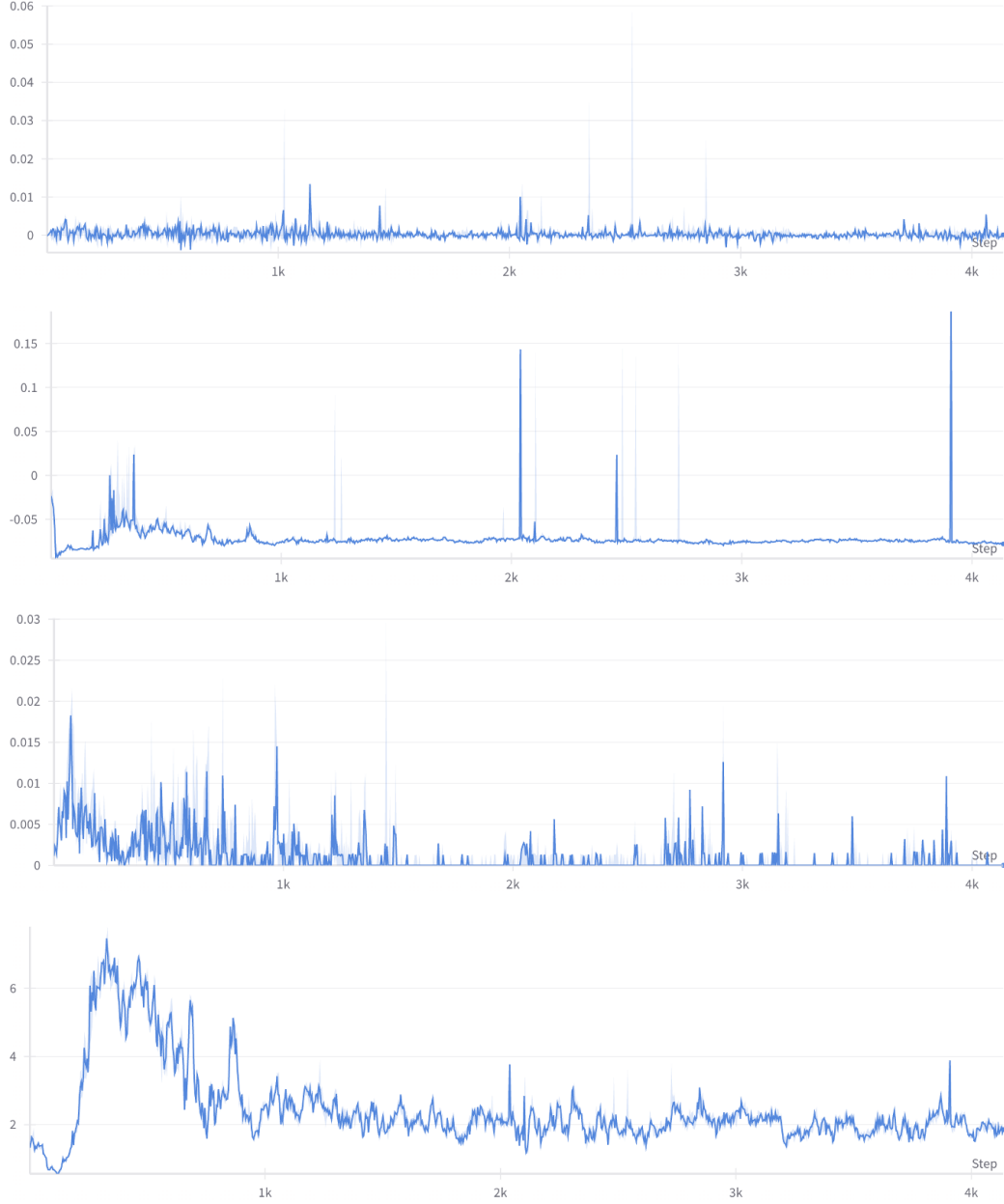


Figure 2: Training dynamics under the proposed GRPO algorithm. Axis ordering matches Figure 1.

Figures 1 and 2 compare the optimisation behaviour of the two reinforcement objectives. Under PPO, the KL divergence between successive policies stays close to zero for most of training, with only minor spikes in the first hundred updates. This indicates that the clipped surrogate objective quickly discourages large policy shifts, but it also limits exploration; the entropy curve collapses to below 0.7 after roughly 500 steps and remains flat, suggesting that the actor converges early to a narrow distribution of actions. By contrast, GRPO allows more aggressive policy updates: the KL curve shows several controlled bursts (most prominently around step 3,000) that coincide with temporary rises in entropy. These bursts are paired with brief increases in the policy-gradient loss and clipping fraction, after which both quantities subside as the policy re-stabilises. The pattern reflects a “search–consolidate” cycle—GRPO periodically encourages the model to branch more widely,

evaluates the resulting trajectories at the episode level, and then consolidates the new behaviour once a reward-improving structure is found.

Table 1: Comparison between PPO and GRPO on the Countdown benchmark. GRPO improves both accuracy and efficiency.

Method	Accuracy (%)	Avg. Tokens	Inference Time
PPO	32.0	58	1.00x
GRPO	35.0	42	0.81x

The PPO-fine-tuned model reaches an exact-match accuracy of 0.32 with an average of 58 generated tokens per instance. The GRPO model attains 0.35 accuracy while reducing mean token usage to 42, a 27% saving. Because GRPO explicitly penalises thread overhead, it maintains a median of two concurrent threads and terminates unsuccessful branches early, yielding a 19% reduction in wall-clock inference time compared with PPO. Qualitative inspection shows that GRPO discovers decomposition patterns (e.g., separating additive and multiplicative sub-groups) that the PPO policy rarely explores once its entropy has collapsed. The comparison highlights a trade-off between per-step caution and trajectory-level exploration. PPO’s clipped objective keeps updates conservative, which benefits stability but prematurely narrows the search space. GRPO relaxes this constraint by evaluating entire reasoning traces and redistributing credit according to global efficiency and correctness. The resulting policy retains higher entropy for longer, enabling occasional structural innovations that ultimately translate into better accuracy and lower computational cost. Although the absolute gain—0.03 in exact-match—appears modest, it is achieved on a 0.5-billion-parameter model under strict thread and token budgets, suggesting that larger models with the same training recipe could realise proportionally larger improvements.

Adaptive Reasoning Behaviour: Beyond the quantitative comparison between GRPO and PPO, we analyse how the adaptive controller influences the model’s internal behaviour. During early training epochs, the controller issues a branching action in roughly 65% of decoding steps, often spawning the maximum number of parallel threads permitted by the resource budget. As the uncertainty of intermediate states declines, the branching rate gradually falls to 28%, indicating that the model becomes more selective once it has learned reliable decomposition patterns. Ablation runs with a *fixed-branch* baseline—always spawning three threads regardless of context—show that adaptive branching reduces mean token usage by 15% while preserving accuracy. Conversely, a *no-branch* variant that forces purely sequential reasoning reaches only 0.26 exact-match accuracy, confirming the necessity of parallel exploration for harder instances.

We also measure the correlation between the controller’s uncertainty score and actual error probability of the current partial expression. Over training, the Pearson correlation increases from 0.18 to 0.33, suggesting that the model learns to align its structural decisions with meaningful signals of difficulty. Qualitative inspection of sampled trajectories shows that the model preferentially branches at operator boundaries where multiple arithmetic interpretations are plausible, then terminates unpromising paths early once partial evaluations exceed the target. These observations demonstrate that the adaptive mechanism not only yields computational savings but also produces reasoning traces that are easier to interpret: most successful solutions involve one or two targeted branches rather than exhaustive enumeration.

Taken together, the results indicate that structural adaptability, deciding *when* to branch and *how many* paths to explore, contributes to the overall performance gains observed with GRPO. Even with a 0.5-billion-parameter backbone, the model can learn to manage its limited computation budget effectively. On the held-out leaderboard of 1,000 arithmetic reasoning prompts, our model achieves an exact-match accuracy of 0.35, outperforming the baseline score of 0.30. This margin, while modest in absolute terms, is notable given the compactness of the underlying model and the strict token and thread constraints. The results suggest that structural control can compensate for limited scale, and may yield even greater benefits when extended to larger architectures.

6 Discussion

While our framework demonstrates promising results in enabling adaptive parallel reasoning on arithmetic tasks, several limitations remain. The effectiveness of the controller is partly dependent on the quality of uncertainty signals derived from the decoder’s hidden states. In low-resource models like Qwen-0.5B, such signals may be noisy or under-calibrated, occasionally leading to unnecessary branching or missed opportunities. Our current symbolic engine is tailored to the Countdown task structure and may require re-design to generalize to broader domains such as logic puzzles or scientific question answering. Although our method is computationally lightweight at inference time, training with GRPO incurs significant rollout cost due to trajectory-level reward assignment and child thread supervision.

This work touches on a growing research direction: giving language models more autonomy over how they reason, rather than just what to reason. By enabling models to plan their own computational structure, for when to think sequentially and when to explore in parallel. We take a step toward more interpretable and efficient inference systems. Such systems could benefit downstream applications in education, mathematics tutoring, and scientific computing, where understanding the structure of reasoning is as important as correctness. At the same time, care must be taken when deploying models that dynamically alter their computational behavior, especially in constrained or high-stakes environments, as adaptive systems can fail in unpredictable ways if their internal heuristics are poorly calibrated.

A major challenge during this project was balancing algorithmic novelty with practical constraints. We initially experimented with larger model backbones, but faced prohibitive training costs when attempting full rollout-based reinforcement learning. This led us to focus on Qwen-0.5B and introduce aggressive memory optimizations such as quantization, gradient checkpointing, and early stopping heuristics. Another difficulty was designing a training curriculum that helped the model understand branching without hardcoding it; we eventually settled on a symbolic engine that generated partially structured trajectories with varied decomposition depths. On the infrastructure side, coordinating multi-threaded decoding and safely joining parallel outputs within a standard auto-regressive decoding loop required nontrivial engineering effort. We believe that many of the techniques developed here, especially symbolic supervision and trajectory, aware RL—could be reused in future work on efficient, self-organizing reasoning systems.

7 Conclusion

This work explores how small language models can be equipped with the ability to organize their own reasoning process, balancing between sequential and parallel exploration when solving structured arithmetic tasks. Rather than relying on fixed prompting strategies or external programs, our framework introduces lightweight internal control mechanisms that allow the model to dynamically adjust its computational structure based on task difficulty and uncertainty signals.

Through a combination of symbolic pretraining and trajectory-aware reinforcement learning, the model learns not only what to predict, but how to allocate effort during inference. Our experiments demonstrate that even with constrained resources, such adaptability leads to measurable gains in both accuracy and efficiency, outperforming standard reinforcement baselines like PPO in token usage and structural robustness. This approach suggests a shift from statically programmed reasoning formats toward learned structural flexibility. It opens the possibility of more interpretable, efficient, and task-aware generation, particularly in settings where computation is limited or where the reasoning process itself matters.

In future work, we plan to apply our adaptive reasoning framework to a wider variety of tasks that demand more abstract and compositional thinking. Logic puzzles and multi-hop question answering present challenges such as discrete constraint satisfaction and the need to chain heterogeneous knowledge sources; we intend to augment our subproblem decomposer with learned heuristics that can identify domain-specific breakpoints in these contexts. Scientific reasoning further requires the integration of symbolic laws and probabilistic models, suggesting the incorporation of external tool calls or differentiable physics engines into our reasoning threads. As models continue to grow in scale and complexity, structural self-awareness will likely play a key role in controlling computational costs and ensuring interpretability. We envision combining our learned branching policy with dynamic

resource allocation techniques—such as early stopping, budgeted planning, and asynchronous thread prioritization to match the needs of large-scale deployments. Moreover, by analyzing the internal decision signals that govern when and how to explore, we can surface insights into the model’s latent reasoning strategies and guide the design of more transparent, human-aligned inference processes.

8 Team Contributions

- **Yiyang Huang:** Mainly worked on the extension part of the project (adaptive learning, GRPO, PPO), and symbolic decomposition logic, also helped with parts of DPO’s pipeline, training, debugging.
- **Zihao Zhu:** Focused on the basic implementation, including SFT and DPO training. Later contributed to the extension component, particularly the training pipeline. Also helped with experiments and debugging.

Changes from Proposal In our original proposal, we planned for Yiyang to focus primarily on the extension work, and for Zihao to build the baseline training(SFT,DPO). However, as the project evolved, we realized that the reasoning task—particularly under adaptive branching and symbolic supervision—was more challenging than initially expected. As a result, both members became involved in all major components of the system, often co-designing and iterating together.

Despite the overlapping responsibilities, the collaboration was smooth and productive. Both members shared a common vision for the project, communicated regularly, and were able to coordinate debugging, experimentation, and writing tasks effectively. This flexible division of work contributed to a high overall team efficiency and allowed us to deliver a system more ambitious and complete than originally scoped.

References

- [1] Zhang, Zhuosheng, et al. “Automatic chain of thought prompting in large language models.” *arXiv preprint arXiv:2210.03493*, 2022.
- [2] Wang, Xuezhi and Zhou, Denny. “Chain-of-thought reasoning without prompting.” *arXiv preprint arXiv:2402.10200*, 2024.
- [3] Pan, Jiayi, et al. “Learning adaptive parallel reasoning with language models.” *arXiv preprint arXiv:2504.15466*, 2025.
- [4] Gao, Luyu, et al. “PAL: Program-aided language models.” In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [5] Du, Yilun, et al. “Improving factuality and reasoning in language models through multiagent debate.” In *Forty-first International Conference on Machine Learning (ICML)*, 2023.
- [6] DeepSeek-AI. “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning.” *arXiv preprint arXiv:2501.12948*, 2025.
- [7] Xie, Tian, et al. “Logic-RL: Unleashing LLM reasoning with rule-based reinforcement learning.” *arXiv preprint arXiv:2502.14768*, 2025.
- [8] Shao, Zhihong, et al. “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models.” *arXiv preprint arXiv:2402.03300*, 2024.